Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Taming Reentrancy through Auditable Language Design

Master Thesis

T. Richards

April 7, 2026

Advisors: Prof. Dr. Z. Su, Dr. H. Sun, Dr. C. Li
Department of Computer Science, ETH Zürich

Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

Abstract

As decentralized finance protocols have grown to secure hundreds of billions of dollars in on-chain value, so too has the cost of failure: despite tens of millions spent annually on third-party audits, formal verification, and bug bounty programs, smart contract vulnerabilities continue to cause billions of dollars in losses. Among these, reentrancy – where an external call re-enters the calling contract before its state updates complete – remains one of the most common and damaging vulnerability classes. Because it is a purely structural property of the call graph rather than a semantic bug, it is uniquely suited to prevention through language design. We present a Rust-inspired smart contract language targeting the EVM that introduces two key safety mechanisms: 1. A compile-time effect system requiring developers to explicitly annotate state modifications. 2. Automatically generated reentrancy guards, with declarative annotations that allow developers to selectively re-enable specific reentrant call paths when required. Notably, the guarantees provided by the reentrancy guards allow the compiler to propagate side effect constraints even across reentrant call paths, which is not possible in any other language without sophisticated static analysis tools. The explicit effect annotations make storage mutation patterns machine-checked and visible at function boundaries, making smart contract code simpler to secure for developers, auditors, and bug bounty hunters. Together, these mechanisms render reentrancy vulnerabilities virtually impossible. We evaluate the practical impact of these features against real-world DeFi exploits (2020-2024) and find that 53/54 reentrancy incidents would have been prevented, the sole outlier being a compiler bug rather than a developer oversight.

Contents

Contents iii

1 Introduction 1

2 Background 2

3 Related Work 3

4 Methodology 4
- 4.1 Key Challenges . . . 4
- 4.2 Core Ideas . . . 4
  - 4.2.1 Reentrancy Protection . . . 4
  - 4.2.2 Side Effects . . . 5
  - 4.2.3 Side Effects in External Calls . . . 7
  - 4.2.4 Reentrancy as a Side Effect . . . 8
  - 4.2.5 Algebraic Data Types . . . 9
- 4.3 Discussion . . . 9
  - 4.3.1 Retrofitting Solidity . . . 9
  - 4.3.2 Retrofitting Vyper . . . 10
  - 4.3.3 Rust as a Base Language . . . 11

5 Implementation 12
- 5.1 Pipeline . . . 12
  - 5.1.1 Lexing, Parsing and Desugaring . . . 12
  - 5.1.2 Linking . . . 12
  - 5.1.3 Type Inference . . . 13
  - 5.1.4 Effect Checks . . . 15
  - 5.1.5 Monomorphisation . . . 15
  - 5.1.6 Code Generation . . . 15

6 Evaluation 16
- 6.1 Reentrancy Protection . . . 16
  - 6.1.1 Performance Overhead . . . 17
- 6.2 Effect Annotations . . . 19
- 6.3 Algebraic Data Types . . . 21
  - 6.3.1 Real-world example . . . 22

7 Conclusion **24**

Bibliography **25**

Dummy Appendix **26**

Chapter 1

Introduction

This page unintentionally left blank.

Chapter 2

Background

This page unintentionally left blank.

Chapter 3

Related Work

This page unintentionally left blank.

Chapter 4

Methodology

4.1 Key Challenges

TODO: what are the exact problems we are trying to solve, why are current tools insufficient, etc.

4.2 Core Ideas

4.2.1 Reentrancy Protection

As we have seen, reentrancy vulnerabilities are one of the most prevalent, yet easily classifiable kinds of exploits. While the exact attack pattern may vary, it must by definition always involve a nested call into a contract that is already mid-execution. Interestingly, most smart contracts do not even require reentrant calls for their intended purpose. In fact, many contracts that have been exploited via reentrant calls did not require reentrant calls to work at all. In many cases, the issue was simply a lack of adequate protection against reentrant calls.

Given the high financial stakes in this environment, it seems clear that allowing reentrant calls by default is not ideal. Most projects do not use reentrancy at all, and thus developers should not be forced to reason about such cases unless they intend to specifically make use of reentrant calls. This does mean that the language no longer has a one-to-one correspondence with the behaviour of the EVM itself. However, this allows inexperienced developers to simcply reason about their own code without needing to worry about the intricacies of the underlying virtual machine.

Existing approaches to reentrancy protection almost always involve adding specific permissions to the entrypoints of the contract. For example, standard practice in Solidity and Vyper is to add a `nonReentrant` modifier to all external functions that do not need to be reentered into. If the default behaviour were simply inverted, we would then expect the entrypoints to be labelled with `reentrant` modifiers where necessary. This is already a vast improvement – any project that does not require reentrancy at all would by definition not have to add any such labels at all, while still being fully protected against reentrancy vulnerabilities.

However, there are still some issues regarding contracts that *do* require reentrant behaviour. A typical contract will not just make one external call, but make many different calls to various other contracts. It may well be the case that one of these external calls is expected to be reentrant, but that doesn't necessarily mean that it would be safe for *every* external call

to then call back into the original contract. Yet with granularity at the level of entrypoints, there is no way to restrict this without the user implementing their own logic on top of the existing protections.
The other natural approach would be to decorate the call site with a `reentrant` label of some kind. Again, this leads to a similar issue - while a reentrant call to one entrypoint might be safe, any other entrypoint might lead to a bug. Thus, it seems clear that being able to map from a specific external call to a specific entrypoint is necessary.
When considering this from a "safety-by-default" perspective, this also makes sense. The developer should be able to allow exactly the behaviour they need, without allowing any further reentrant calls to be made.

TODO: probably good to insert a figure clarifying the three different approaches mentioned: labelling entrypoints, labelling calls, our solution (i.e. restricting to single call path)

Our solution

We propose the following approach: All external calls are not able to reenter the contract by default. The annotation of reentrant behaviour is made at the *external call* site, and must specify the *exact* entrypoint which is intended to be reentered.
For example, let's say we are making an external call to `address` with calldata `data`, and this call should be allowed to reenter the `repay(...)` function. This could be specified as follows:

```
EVM::reentrant_call(address, data, Self::repay)
```

Contrast that to a regular external call, which would be:

```
EVM::call(address, data)
```

The syntax restricts the reentrant call path to exactly what the developer would expect during normal execution. It does not impose much overhead – we simply add a reference to the corresponding entrypoint, which the developer surely has in mind anyway. Additionally, the control flow becomes much more obvious: If we consider an auditor reading through the code line-by-line, it is immediately obvious that the above call might lead to a call back to the `Self::repay` function. However, if only the entrypoint itself were annotated, then the transfer of control flow would not be clear.

4.2.2 Side Effects

One might say that the entire purpose of smart contracts is to produce side effects on a blockchain. Thus, considering side effects and their consequences is clearly a central part of reasoning about smart contract safety. This is especially relevant for smart contract auditors – given a codebase they have never seen before, they are tasked with ruling out bugs and vulnerabilities within the project, often within only a few weeks. Thus, it is vital for them to be able to reduce the search space as much as possible. We have already shown how this can be done by removing all non-necessary reentrant calls, but the same logic is also valid for side effects.
For illustration, consider something as simple as this:
`f(g(...), h(...))`
What can we say about the side effects produced by this expression? Clearly, `f` must be executed last, since it relies on the results of the other calls. However, the evaluation order

of `g` and `h` is ambiguous. Of course, it depends on the language specification – but ideally an audit-oriented language would prevent ambiguous syntax in the first place.
In the specific cases of Solidity and Vyper, the situation is even worse: Not only is the syntax ambiguous, the evaluation order is actually *undefined* in such cases! That is, it is up to the compiler to choose whatever order it deems most convenient. While the evaluation order is almost always left-to-right in practice, there are some exceptions, as demonstrated by the winning submission to the **Underhanded Solidity Contest 2022**. Of course, this specific behaviour is mostly limited to code that is designed to be misleading. However, such intentionally hidden bugs can be just as dangerous as unintended ones.
In this work, we have chosen to focus specifically on *state modifications*. They are likely the most relevant type of side effect for this application, but the principles used can be generalized to many other classes of side effects, such as logging (event emission), state reads, exceptions (reverting of transactions), etc.

Our solution

Our guiding principles in this case were the following:

- There should be no hidden or implicit side effects.
- The ordering of effects must always be unambiguous.

Let's first look at an illustrating example.

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) {
    let val_1 = self.n;

    self.increment();

    let val_2 = self.n;
    assert(val_1 == val_2);
  }
  fn increment(self) {
    self.n += 1;
  }
}
```

Here, we can see that there is a "hidden" side effect when calling `self.increment()`. This may result in the incorrect assumption that `self.n` does not change during this call. While it may be obvious in such a simple example, when considering internal function calls in general, it's not so simple. In a sense, the side effects are "hidden" inside the implementation details of the function being called, so we cannot know the possible side effects without first fully understanding its implementation.
There are two issues at play here – let's first consider the hidden nature of the side effect. If we want to avoid such cases, the developer must somehow specify that `self.n` may change when `self.increment()` is called. There are many potential ways of doing this, but for now let's simply surround the relevant statement with a *permission block*:

```
    modify(self.n) {
```

```
      self.increment();
    }
```

Note that we would not need this decoration for *direct* side effects, such as the `self.n += 1;` statement later on. This is because the side effect is visible in the syntax, it is not hidden. This still leaves us with the second issue – when calling a function, we may not know all potential side effects that might be induced. The caller should not be required to understand the precise implementation details of a function. All relevant properties of a function should be provided by its signature. We can amend this by requiring the `increment` function to declare that it might modify `self.n`. That way, we encapsulate the information about potential side effects behind the signature of the function.

```
  fn increment(self) modifies(self.n) {
    self.n += 1;
  }
```

In this case, we have used a different keyword for the function-level declarations as opposed to the block-level permissions. This is mostly to make clear that they do not mean the same thing – `modify` is used to annotate *indirect* side effects occurring within the body of a function, while `modifies` is an element of the function signature, declaring the possible side effects that may occur when the function is executed. Note that is likely that this choice of syntax would be confusing to new developers. However, the main purpose here is to demonstrate this as a proof of concept, the specific syntax could always be changed.

4.2.3 Side Effects in External Calls

There is one obvious limitation to dealing with side effects in this way – what happens if we don't know the details of the function being called? This is an unavoidable situation when a contract makes an external call into untrusted code. As soon as the control flow transfers to arbitrary code, the compiler is unable to enforce its own rules regarding side effects. This makes it impossible to enforce properties about *external* state. In fact, it is even difficult to reason about *internal* state at all, due to the possibility of reentrant calls. This is where our first core feature comes to the rescue.

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) modifies(self.n) {
    let addr: Address = EVM::caller();
    let data: Bytes = Bytes::empty();
    modify(self.n) {
      EVM::reentrant_call(addr, data, Self::increment);
    }
  }
  pub fn increment(self) modifies(self.n) {
    self.n += 1;
  }
}
```

As it turns out, we have all the information we need – every external call site must specifically declare which functions it may reenter. As we saw before, every function must now declare its possible side effects. These two facts let us guarantee which internal side effects may happen during any external call *at compile time*. This type of guarantee would not otherwise be possible without the use of sophisticated static analysis to rule out reentrancies.
Thus, it becomes even simpler to reason about reentrancy vulnerabilities. Not only do we know exactly what entrypoint is being called, it is also immediately clear at the call site which state variables might be modified in the nested call, despite the external call executing arbitrary code. In fact, if the side effect declarations were extended to also include state reads, we could also much more easily rule out read-only reentrancy bugs as well.

4.2.4 Reentrancy as a Side Effect

Ignoring state modifications for a moment, consider the following example:

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) {
    let val = self.n;
    self.bar();
    self.n += 1;
  }
  pub fn bar(self) {
    EVM::reentrant_call(addr, data, Self::foo);
  }
}
```

As previously mentioned, we want to avoid hidden side effects in the code. There is another hidden effect at play here – from the perspective of the `foo` function, it is not clear that it may be called in a reentrant context. This is an especially dangerous situation, since by definition there will be a possible overlap between the effects produced by the reentrant call and those in the top-level call. Even if the reentrant call were made to some other entrypoint, it is still vital to carefully consider what exactly may happen during the reentrant call. Therefore, we treat reentrant calls the same way as the state modifications – every function signature must declare its possible reentrancies, and any indirect reentrancies within a statement must be surrounded by a permission block.

```
  pub fn foo(self) reenters(Self::foo) {
    let val = self.n;
    reenter(Self::foo) { self.bar(); }
    self.n += 1;
  }
  pub fn bar(self) reenters(Self::foo) {
    EVM::reentrant_call(addr, data, Self::foo);
  }
```

The overhead induced by this requirement is quite small compared to the state modifications, since reentrant calls are much less common. But we will discuss the overhead of the effect

annotations in more detail later on.
There is one edge case we have neglected to address so far. How can we deal with reentrancies that make further reentrant calls? What about a recursive function reentrantly calling itself? In general, this is an exceedingly rare occurrence. In fact, it seems unlikely for any use case to require nested reentrancies at all. Thus, it would be reasonable to simply disallow this case from occurring at all. Even so, there is no real barrier to reasoning about nested reentrancies with the same logic we have used so far. The caller of such a function must simply declare all the possible further reentrancies, which may include the calling function itself.

4.2.5 Algebraic Data Types

TODO

4.3 Discussion

To briefly summarize, the three core features we propose are:

- Specifying exact call paths for reentrant calls.
- Explicit side effects (state modifications, reentrant calls).
- Algebraic Data Types

There are several ways these features could be made available to developers as conveniently as possible. The most obvious solution would be to modify an existing widely-used language such as Solidity or Vyper. However, this would present significant challenges, as we will see in a moment. Most other projects targeting EVM are significantly out of date, and often have their own unique feature sets which do not compose well with ours. Additionally, there is little benefit in modifying a language that is not already being used for smart contract development, since the language will have been designed with other goals in mind. Unfortunately, that seems to indicate the necessity of designing a new language which specifically fits our desired features. In the interest of making it easy to adopt, it should be based closely on an existing well-known general-purpose programming language. As you may have surmised given the code examples we have given so far, the language chosen in this case is Rust.

4.3.1 Retrofitting Solidity

Solidity is the oldest, most widely used programming language for EVM. As such, there is also a large incentive not to break backwards compatibility. While there have been breaking changes in the past, they are quite rare. In fact, since the release of Solidity v0.8 in December 2020, no more breaking changes have been made at all.

ADTs

Adapting Solidity to allow ADTs would not present too much of a challenge. Assuming they were handled similarly to Rust, one could simply extend the concept of a Solidity `enum` to include fields. Although one might have to reconsider the syntax of a `switch` statement, or alternatively add some new syntax to enable variable bindings when pattern matching. Of course, great care needs to be taken to ensure backwards compatibility not

only in the syntax, but also in the ABI encoding of types and function calls, since newly deployed contracts should still be able to interact with all existing contracts.

Reentrant Call Paths

As long as we only require reentrant external calls to declare their corresponding entrypoints, this is again just a small syntactic change to the language. However, the `reenters(...)` declaration in function signatures does not play well with Solidity's contract inheritance and virtual functions. This is not necessarily a deal-breaker, but does present some real obstacles. For example, how could one define an overridable function in an abstract contract, such that it is allowed to reenter a function in an implementing contract? There are solutions, but they are all work-arounds of some kind. Additionally, one would have to be able to differentiate between overloaded functions, and between the functions defined on the current contract versus those on its super-types (i.e. `reenters(super.foo)` vs. `foo`).
What's more, Solidity also features function pointers. One would either have to prevent creating pointers to reentrant functions entirely, or extend the function pointer type to also include potential reentrancies. However, it would not be generalizable – if we wanted to pass `function() reenters(this.foo)` and `function() reenters(this.bar)` as arguments to the same function, we would need to be able to generalize over the function to be re-entered. Solidity has no notion of generics or type variables, so this would not be a natural extension to the language.

Effects

Similar to the reentrancy annotations, the main obstacles here are provided by contract inheritance, as well as function pointers. The difficult questions that would need to be answered are:

- How can an abstract contract specify a modification to a state variable of the implementing contract?
- How can we extend the notion of function pointers to naturally include their effects?

Again, we believe these to be prohibitively large obstacles to extending Solidity with our proposed features.

4.3.2 Retrofitting Vyper

The design of the Vyper language is much simpler by comparison. It is closely based on Python, but intentionally restricts its feature set so as to remain as simple as reasonably possible. Due to its widespread use, the language is similarly tied up with the responsibility of maintaining backwards compatibility, although less so than Solidity. Thanks to the simplicity of the language, there are fewer obstacles to implementing our proposed features in the language. However, the philosophy behind Vyper (and Python) is more about making the language easy to use, and reducing developer overhead. Furthermore, Vyper restricts access to low-level EVM features such as memory allocations, which makes it tricky to reduce the memory footprint. For that reason, it seems unlikely that such features would ever be adopted by the Vyper compiler team. Even so, we will briefly discuss the viability of each feature in detail.

ADTs

There is no real barrier to implementing ADTs and pattern matching in Vyper, since Python already has support for such types. One slight issue is that Vyper does have reference types, but when passing these types to functions they are implicitly copied. This incentivizes less abstraction and shallower call hierarchies. As such, top-level code that involves many ADTs would be littered with pattern matching case distinctions, rather than "hiding" such functionality within methods defined on the type itself. This makes the feature much less attractive to use, especially since many of Vyper's advantages are provided by its simplicity and straight-forward code. Furthermore, Vyper does not give the developer precise control over when memory allocations happen. Due to the frequent copying of memory, as well as the larger memory footprint induced by ADTs, it would likely be tricky to keep allocations below a reasonable limit.

Reentrant Call Paths

Although Vyper does not prevent reentrancies by default, it is likely for this to be changed in the near future. However, due to the desire for simplicity, this will be handled via a `@reentrant` modifier on the entrypoints. For this reason, it again seems like the restrictive nature of our feature set is not a good fit for Vyper, even though there are not necessarily significant barriers preventing such an implementation.

Effects

Vyper provides modularity through composition. This avoids the problematic function overriding and overloading we encountered with Solidity. The language also does not feature function pointers, which was the other major road-block. Thus, the annotation of side effects would actually fit quite well into the language design, especially since it already has annotations for function declarations. However, *requiring* the side effect annotations to be provided by the developer would go against the language's philosophy, and present a huge breaking change to all existing code. Thus, it would be exceedingly unlikely for this kind of change to be adopted or well-received by existing users of the language.

4.3.3 Rust as a Base Language

As demonstrated by Vyper, it can be advantageous to design a niche language closely on an existing general purpose language, so as to ease adoption for new developers. Initially, we had planned to design a functional language, likely syntactically similar to Haskell. However, the decision was made early on to instead make the language imperative, since functional languages tend to be more difficult to learn and are not widely used. Rust was the next natural choice, since it features ADTs and pattern matching natively. In fact, there are not many mainstream imperative languages that do support pattern matching. The somewhat low-level nature of Rust also is a benefit, since we do want to give the developer control over what operations are happening, most notably memory allocations and storage reads / writes. It is of course still possible to build up many layers of abstraction, for example by defining methods on user-defined types or using traits.

Chapter 5

Implementation

The implementation of our compiler consists of roughly 30'000 lines of Rust code, as well as another $\tilde{1}$0'000 lines of code for tests. The pipeline is a of relatively standard design, with the novel addition of a pass to check the effect annotations. All in all, it consists of 13 steps, although some similar functionalities are split up simply because not all the necessary information is available the first time around. For example, syntax desugaring is applied once before type inference, and again afterwards – this is an unfortunate consequence of the type system, since it is not clear right away whether an expression such as `Type::Member` refers to a constant, function, or enum variant (we do not support associated types). Moreover, the member may be present in multiple different traits or `impl` blocks, so until type inference occurs we can't know which one it refers to. This is especially relevant for pattern matching, since matching on a constant is very different from matching on an enum variant. Each step in the pipeline is explained in more detail in the following section.

5.1 Pipeline

The pipeline consists of the following (unique) steps:

- Lexing
- Parsing
- Desugaring
- Lexing
- Linking
- Type Inference
- Effect Checks
- Monomorphisation
- Code Generation

5.1.1 Lexing, Parsing and Desugaring

5.1.2 Linking

The linking is executed in two phases. First, all non-local definitions are collected and stored according to their module hierarchy. Then, we can pass through the AST a second time,

resolving each reference to its appropriate definition. It is not always possible to uniquely resolve every reference at this stage, since we are still lacking type information, as previously mentioned. Therefore, any expressions involving trait members or similar are resolved as far as they can be, then stored for later inference.

References to local variables are much simpler to resolve, since we can simply visit the AST in order and simply keep track of which variables are in scope.

5.1.3 Type Inference

The type inference is likely the most complex element of the compiler. The issue is a typical chicken-and-egg situation – not all references can be resolved prior to type inference, but the types of those references cannot be known until they are resolved. Furthermore, soundly dealing with generics and trait bounds is a tricky business, since such constraints must be propagated and subsequently checked to be valid. We implement a version of Hindley-Milner type inference, loosely according to the approach described in *Typing Haskell in Haskell* [**?**], an excellent paper describing how a relatively similar type system can be soundly inferred. The main difference is that Haskell's class definitions differ slightly from Rust's traits, in that they do not take type arguments. That is, the type implementing a Haskell class may take type arguments (such as with Haskell's Monad class), but the class itself does not directly constrain the argument type. By contrast, Rust traits explicitly take type arguments, and even may constrain them to fulfill certain trait bounds. This mainly changes how one must detect and deal with overlapping trait instances.

We first define a few notational conventions.

Symbolic Types Since we are dealing with parametric polymorphism (generics), we also necessarily introduce the concept of symbolic types. Moreover, we need to be able to express trait bounds. The notation `<T: Foo> T` essentially means "some type T that implements the trait `Foo`".

Sub-Typing We define the sub-typing relation between types, $T_1 <: T_2$, meaning that T_1 is a sub-type of T_2. Note however, that this sub-typing relationship differs slightly from what you might be used to from object-oriented languages. Rust has no inheritance, instead what is meant by this is that a type is "more specific" than another. For example, the type `<T: Integer> T`, meaning some symbolic type T that implements the trait `Integer`, is "less specific" than the type `u256`. Intuitively, this is because T could refer to any type that is an integer, whereas `u256` refers to a specific integral type. In general, $T_1 <: T_2$ holds if there is some set of substitutions σ to the bound type variables in T_2 such that $\sigma(T_2) = T_1$.

Unification and Matching There are two important operations on types for our type inference. The first is the *most general unifier* (*mgu*). Given two types A and B, $mgu(A, B)$ is defined to be the "most general" type T such that $A <: T$ and $B <: T$. "Most general" means that for *any* type T' with $A <: T$ and $B <: T$, then $T' <: T$ must also hold. Therefore, T is at least as general as any other type unifying A and B, thus making it the most general unifying type.

The second operation is matching one type to another. For it to yield a valid result, $B <: A$ must hold. The goal is to find a set of substitutions σ, such that $\sigma(A) = B$. For example, matching `<T: Integer> T` to `u256` would give the substitution $T \rightarrow$ `u256`. This operation is useful when assigning one value to another, since the left-hand-side of an assignment must

have a more specific type than the right-hand-side. For example, the following assignment *is* valid:

```
let x: u256 = Default::default();
```

Whereas this one is not:

```
let x: T = 0u256
```

In the first example, the right-hand-side is simply some symbolic type `<D: Default> D`, and thus can be matched to `u256`. In the second example, while T is symbolic, it could refer to any type and may not be constrained.

Inference

The general approach to our type inference is to assign each expression a fresh symbolic type. Then, whenever two symbolic types are combined to form a compound expression, they can be unified according to the constraints induced by this expression. For example, let's define two variables `a` and `b` with symbolic types A and B. To infer the value of the expression `a + b`, the syntax must first be desugared. The types must implement the `Add` trait, therefore we can safely rewrite the expression to `Add::add(a, b)`. Note that `a.add(b)` is *not* a safe rewrite, since the type A may also have other methods with the same name. Now, we can check the trait's definition to see what constraints must apply to A and B for this to be valid. Let's say the definition of the method is as follows:

```
fn add(self, other: Self) -> Self
```

We can infer that A must equal B, as well as that A must implement the `Add` trait. Thus, we infer the type of `a + b` to be `<A: Add> A`. This trait bound must be propagated until A can be concretized, or until it is unified with a generic type with the corresponding trait bound.

Alternatively, there may be more indirect ways of satisfying the bound. For example, if we find that A implements trait `Foo`, but that `Foo` lists `Add` as one of its super-traits, then the bound can be dropped as well. Alternatively, there may be some instance of the `Add` trait that applies to A, even if the type is not yet concrete. This is often the case for types such as `Option<T>`, since we can define addition over the `Option` type by requiring `T: Add`.

Recall the important distinction between *unifying* types and *matching* them. This becomes especially critical when dealing with variable assignments and inferred types. In an assignment such as `foo.x = y;`, we cannot safely modify the type of the left-hand-side, even if it is not a concrete type. The main exception is if we have a novel variable declaration with an inferred type, such as `let x: _ = y;`. Here, we assign `x` a new symbolic type, but keep track of the fact that it stems from a type that should be inferred, meaning it can safely be unified with RHS types.

Method Resolutions

The trickiest part of the type inference is to correctly resolve invoked methods to their definitions. As previously mentioned, the resolution requires type information (e.g. to know if a type implements a certain trait), but the type inference requires knowing which method is being referenced. Therefore, we cannot necessarily reject ambiguous method references right away. Instead, we run through the entire type inference once, inferring as many types as possible. If there are still unresolved symbolic types, we revisit them to see

if the methods can now be uniquely resolved. If there is still an ambiguity at this point, we reject the program as being invalid (although it may still have a uniquely correct type inference, but this should be exceedingly rare).

5.1.4 Effect Checks

5.1.5 Monomorphisation

5.1.6 Code Generation

Chapter 6

Evaluation

6.1 Reentrancy Protection

Clearly, the main goal of providing protection mechanisms to guard against reentrancies is to prevent real-world vulnerabilities. Thus, it is vital to consider how our approach would have impacted historical cases where reentrancy vulnerabilities led to an exploitable security flaw. To analyze this, we can use the vulnerability dataset introduced in the background chapter – it contains 54 cases of reentrancy exploits, totalling $187 million in lost funds. These cases can be further broken up into a few categories. Firstly, the majority of exploited contracts did not make use of reentrant behaviour at all, instead they simply did not adequately protect against it. For these 37 cases, simply rejecting all reentrant calls by default would have sufficed.

Next, there were 6 cases of read-only reentrancies – here, the contracts being exploited were not being reentered, instead they relied a price oracle to determine a conversion ratio between two currencies. However, the oracle contracts did not protect their read-only functions against reentrant calls. Thus, the price could be manipulated for the duration of the reentrant call, and when the oracle price is read from, an incorrect value is returned. This results in the unfortunate situation where the contract being exploited is not really implemented incorrectly, instead it simply relied on an unreliable oracle. Because this type of exploit was first discovered in 2022, it is only recently that it has become standard to protect read-only functions against reentrant calls. In fact, applying the standard `@nonReentrant` modifier to a view function in Solidity is not possible, since the modifier itself writes to storage. Similarly, it was originally not possible to use the built-in @nonreentrant modifier on view functions in Vyper. Thus, we argue that these vulnerabilities were the result of a lack of appropriate tooling, as well as the knowledge that this type of vulnerability is even possible. Once again, this issue would have been avoided if the oracle contracts had simply rejected all reentrant calls.

The next category involves certain token standards. There are various token standards, such as ERC-777, ERC-677, and more, which mandate a `beforeTokenTransfer` (or similar) call to the receiver of a transfer, so as to give the receiver a chance to reject the transfer. However, as these standards are extensions to the ERC-20 token standard, they can be used in place of any other ERC-20 token. Thus, any protocol that involves ERC-20 tokens must carefully consider the possibility that a token they interact with might lead to reentrant calls. Another 6 of the reentrancy vulnerabilities were due to this oversight. While it is true that rejecting all reentrant calls might suffice in this case, it does also mean that interacting with some of the mentioned token standards would no longer be possible, due to the mandated

callbacks. Since the creators of these protocols did intend to interact with such callback tokens, it is by definition part of the use case they were intending. Thus, we argue that a more sophisticated mechanism would have been necessary to prevent these vulnerabilities.

Two of the vulnerabilities involved contracts with flash loan functionality. The usual way this works is the following: The external user calls the flash loan contract, specifying an address to a contract they control, as well as an amount of some currency to borrow. Next, the flash loan contract transfers this amount to the specified contract, then makes an external call to it. When this external call terminates, it asserts that the full balance has been repaid, along with some fee on top. Sometimes, the user is expected to make a reentrant call to a `repay` function on the flash loan contract, although this is not strictly necessary. In any case, it becomes extroordinarily important to handle reentrant calls correctly - if borrowing some funds sets the users debt to the amount borrowed, then reentrantly borrowing a second time would overwrite this value. Furthermore, correctly tracking the balances to assert that the full debt has been repaid is again critical. Flash loans in general are extremely prone to being drained of funds, since the use case itself inherently involves transferring a large amount of funds to an address the user controls. For example, in the specific case of the DFX exploit, the same contract could also be used to simply deposit funds and withdraw them. However, the flash loan functionality simply checked the balance of the contract as a whole to see if the borrowed funds had been returned. Moreover, the `flash` function was the sole entrypoint without a reentrancy guard. Thus, the user could borrow the funds without triggering a reentrancy guard, then reentrantly deposit the funds regularly. The contract then saw that the funds had been returned, and erroneously assumed they had simply been transferred. Finally, the user could then withdraw to completely drain the contract of funds.

A further two vulnerabilities involved manually written reentrancy guards that intended to allow some reentrant functionalities but disallow others. Clearly, this use case required reentrant calls, but the adequate tools were not available to do this correctly. Hence, we again argue the importance of being able to granularly specify reentrant call paths.

The final case involved a bug in the Vyper compiler. The built-in reentrancy guards were not implemented correctly in some versions, allowing them to be bypassed. While it is true that rejecting reentrant calls would have prevented this, it is unfair to argue that the issue could have been addressed by language design. The developers wrote secure code, it was simply compiled incorrectly. Hence, we do not claim that our tools would have prevented this exploit.

These findings are summarized in table 6.1, where "Safety by Default" refers to whether or not the vulnerability would have been prevented by simply rejecting all reentrant calls, and "Path-Specific" refers to whether our proposed protection mechanism of specifying the reentrant call path would have sufficed.

6.1.1 Performance Overhead

Of course, no runtime safety checks come without a tradeoff. In this case, it's even quite costly – storage operations are one of the most expensive features of the EVM. However, offloading the responsibility of reentrancy guards to the compiler gives us an advantage: Instead of simply adding the same @nonReentrant modifier to every entrypoint, we can choose the cheapest approach at compile time. For example, a function that makes no external calls does not need to *write* to the guard variable, it simply needs to *read* it. The main downside we have compared to standard reentrancy protection is read-only functions.

Category	Count	Funds Lost	Safety by Default	Path-Specific (Ours)
Missing Guard	37	$151 million	Yes	
Read-only Reentrancy	6	$9.6 million	if oracle contract uses mechanism	
Callback Token	6	$57.6 million	incompatible	Yes
Manual Guards	2	$27.7 million	No	Yes
Flash Loan	2	$5.7 million	No	Yes
Compiler Bug	1	$41 million	Unclear	
Total	54	$187 million	43/54	53/54

Table 6.1: Preventability of 54 real-world reentrancy vulnerabilities (2021-2024)

In most cases, they are not guarded, meaning the guard variable is never accessed at all. Thus, if a transaction calls exactly one read-only function of the contract, we will pay the extra one-time cost of loading a cold storage slot. Any further accesses will be cheaper, but still induce the slight extra cost of loading a warm (cached) storage slot. The expected difference in gas costs can be seen in table 6.2.

Cache State	Scenario	Standard	Ours	Delta
Warm	Default	300	300	0
	No external calls	300	100	-200
	Read-only	0	100	100
Cold	Default	2300	2300	0
	No external calls	2300	2100	-200
	Read-only	0	2100	2100

Table 6.2: Differing gas costs of reentrancy protection mechanisms

To verify these differences and put them in context, we measured the gas costs of some simple ERC-20 functions in the various test cases. Note that ERC-20 tokens do not usually require reentrancy protection mechanisms, we simply chose them as they are by far the most ubiquitous type of contract, and ERC-20 transfers are the most common kind of transaction.

A 'warm' ERC20 token transfer was measured to cost the following:

- Unguarded: 10'621 gas
- Standard guard: 10'977 gas
- Our guard: 10'843 gas

As we can see, we are saving 134 gas on this call, out of the expected 200. This is mostly due to the extra arithmetic operations in our reentrancy guard, since it must extract the bit

associated with the specific entrypoint before checking it. It may also be the case that the Solidity compiler is not able to optimize the stack operations as much, since we are restoring the original value of the guard once the call terminates. Thus, any internal function call must clear the stack when it exits, rather than being able to directly return (discarding anything left on the stack for free).

A 'cold' ERC20 total supply check (read-only) was measured to cost:

- Unguarded: 2248 gas
- Our guard: 4489 gas

Here we can see the real cost of loading an extra storage value. Not only are we paying the cost for the runtime check that is usually omitted, we also pay for loading a storage slot that would not otherwise have been accessed. This results in the 2241 extra gas in our call. Note that it is not possible for the `totalSupply` storage slot to be warm while the reentrancy guard is cold, since our approach accesses the guard on every call. Additionally, the base gas cost of a transaction is 21'000 gas. Thus, even if we assume a contract function which costs nothing and instantly returns, the absolute worst case scenario, we pay at most 10.7% more gas. While this does seem expensive, it can occur at most once per transaction, and most transactions are significantly more expensive. However, we do acknowledge that there is a tradeoff at play – is it worth the extra security guarantees to pay a small additional cost on every transaction? As we have seen, read-only reentrancy vulnerabilities have lead to at least $9.6 million dollars in lost funds, if not more. Assuming a conversion of 20'000 gas to $\approx$ $0.01, this would be enough to pay for roughly 10 billion extra cold storage loads. Hence, we argue that the tradeoff is worth it as default behaviour, although developers might want to opt out of these extra checks in favor of cheaper transactions.

6.2 Effect Annotations

The primary goal of the effect system is to enhance auditability of code. However, a secondary advantage is that developers are required to double-check their work. If they write annotations, but the code induces different effects, this will be reported by the compiler. The compiler can even produce warnings when effects are annotated, but not produced by the code! Thus, any unexpected or missing side effects will be flagged, letting the developer know that something is awry. Furthermore, the tedious nature of writing such annotations will incentivize an approach to development which reduces the number and diversity of side effects. This is a desirable outcome, since this necessarily entails a smaller potential for unintended interactions between effects. Since the effects become simpler to reason about, this will also accelerate the auditing process.

For auditors, the explicit nature of these annotations has two further advantages: As mentioned, they demonstrate the developers intent when writing the code – thus also serving as a form of documentation for the contract under audit. More importantly, the modularity of the effects, i.e. that they are individually declared for each function, allows the auditor to focus on reasoning about a single function at a time, rather than first having to consider which side effects may occur in nested calls.

The clear downside of requiring all side effects to be annotated, of course, is the extra time it takes to write these annotations. Unfortunately, auto-generating these annotations would defeat some of the purpose – the developer would not be double-checking their work, and likely would just ignore the annotations entirely. For auditors, it would still

be useful, although no more useful than the output of any static analysis tool, with the exception of reentrant calls. Due to the native modelling of reentrancy, our compiler is able to directly propagate side effects (or lack thereof) across reentrant calls. This is not possible with any other language, without first running sophisticated static analysis to determine which reentrant calls are even possible.

The key question for developers is of course the following: How many extra lines of code do they need to write? This of course depends heavily on the code architecture. However, we will show that many well-known projects already follow an "effect-oriented" approach to development, meaning there is little additional effort to write annotations, let alone design the code in such a way to minimize their number. For the analysis of the number of additional lines of code, we will use 5 real-world smart contracts, with various levels of complexity:

- ERC-20 Token: 50 LoC
- Simple Token Exchange: 300 LoC
- AAVE Lending Pool Core: 1100 LoC
- Curve Tricrypto: 1500 LoC

For each contract, we count the number of state-modifying functions, as well as the total number of indirect side effects within those function bodies. This lets us estimate the total number of required annotations, and hence give us an approximate overhead when compared to the contract's total lines of code. Of course, this does not necessarily measure the extra *time* required to write the correct annotations, since that might require careful consideration. Still, it gives us an approximate fraction of additional effort that would be necessary due to the effect system. The results of these measurements are detailed in table 6.3.

Contract	State Vars	LoC	Effectful functions	Indirect Effects	Additional LoC
ERC-20 Token	3	50	6 / 9	2	16.67%
Simple Token Exchange	8	300	7 / 16	2	9%
AAVE Lending Pool	5	1100	42 / 88	32	6.73%
Curve Tricrypto	24	1500	33 / 90	24	3.8%

Table 6.3: Additional lines of code required by effect annotations

What we can clearly see is that there is an inverse proportionality relationship between code size and overhead percentage – the larger a contract becomes, the less the annotations contribute to its overall sizer in terms of lines of code. Complex contracts don't necessarily have many different state variables that are modified in many different combinations – instead, they are often grouped together and modified in similar ways. Therefore, the code dealing with state modifications does not scale in line with the complexity of the contract, instead it grows at a slower rate. Of course, the sample size here is not very significant, it merely serves to indicate the approximate percentages and the inverse relationship.

6.3 Algebraic Data Types

When modelling a state machine, it is standard practice to make use of regular enumeration types. However, such a value can only encode a single state, but not contain any auxiliary information. Standard workarounds include using "magic values", i.e., encoding state information via setting a field to a specific predetermined value, as well as defining a "kitchen-sink" struct, namely one that contains all fields for all states at the same time.

For example, ERC20 tokens are specified such that the value `u256::MAX` $= 2^{256} - 1$ represents an unlimited allowance, whereas any other value represents an allowance equivalent to its actual quantity. Now, any read or write to such an allowance must be checked against this magic value, leading to a higher risk of developer mistakes. Here, we compare the "standard" implementation of the ERC20 `transferFrom` function, as specified by the ERC20 token standard:

Listing 6.1: Solidity

```
contract ERC20 {
  mapping(address => uint256) private _balances;
  mapping(address =>
    mapping(address =>
      uint256)) private _allowances;

  function transferFrom(
    address from,
    address to,
    uint256 value
  ) external returns (bool) {
    require(from != address(0));
    require(to != address(0));

    address spender = msg.sender;
    uint256 amount = _allowances[from][spender];
    if (amount < type(uint256).max) {
      _allowances[from][spender] = amount - value;
    }

    execute_transfer(from, to, value);
    return true;
  }
}
```

Listing 6.2: Ours

```
contract ERC20 {
  balances: HashMap<Address, u256>,
  allowances: HashMap<Address,
    HashMap<Address, Allowance>>,
}
impl ERC20 {
  pub fn transfer_from(self,
    from: Address,
    to: Address,
    value: u256
  ) -> bool {
    assert(from != Address::ZERO);
    assert(to != Address::ZERO);

    let spender = EVM::caller();
    self.allowances[from][spender].spend(value);
    self.execute_transfer(from, to, value);
    return true;
  }
}
enum Allowance {
  Amount(u256),
  Unlimited,
}
impl Allowance {
  fn spend(self, value: u256) {
    match self {
      Allowance::Amount(amount) => {
        amount -= value;
      }
      Allowance::Unlimited => (),
    }
  }
}
```

Note: In both cases, no underflow check is necessary, because both compilers insert overflow checks on arithmetic operations by default. However, it may sometimes be preferable to check manually in order to produce a more specific error message.

Defining an ADT in such a manner provides two distinct advantages: First, it becomes much simpler to separate the underlying logic (reducing the allowance by the spent amount) from the contract's functionality (transferring tokens). Second, the type system now makes it impossible to misinterpret an unlimited allowance as a finite one, whereas in Solidity the developer needs to actively remember to check for the "magic value" every time they use an allowance.

Of course, in such a simple example the benefits are dampened by the overhead induced

by the additional code needed to define the ADT. However, in more complex cases the advantage becomes much clearer. Furthermore, smart contracts often model processes that can be easily represented by a state machine. This is the perfect application for an ADT, as we will see in the next example.

6.3.1 Real-world example

The widely-used OpenZeppelin contract library features a `Governor` contract, which models voting processes. Users can submit proposals, each one of which can be in one of eight different states. Depending on the state, a proposal needs to keep track of different values. For instance, while voting is active, it needs to store the deadline for the end of the voting period, as well as the actual vote counts. This data is stored in the `ProposalCore` struct. Precisely which `ProposalState` the proposal is currently in is dictated by the values stored in the fields. The logic of determining this state is implemented by the `state(...)` function.

Listing 6.3: Proposal State Data

```
struct ProposalCore {
  uint48 voteStart;
  uint32 voteDuration;
  bool executed;
  bool canceled;
  uint48 etaSeconds;
}
enum ProposalState {
  Pending,
  Active,
  Canceled,
  Defeated,
  Succeeded,
  Queued,
  Expired,
  Executed
}
```

Listing 6.4: Proposal State Logic

```
function state(uint256 id) returns (ProposalState) {
  if (proposals[id].executed) {
    return ProposalState.Executed;
  }
  if (proposals[id].canceled) {
    return ProposalState.Canceled;
  }

  uint256 start = proposals[id].voteStart;
  uint256 now = clock();
  if (start >= now) {
    return ProposalState.Pending;
  }

  uint256 deadline = start + proposals[id].voteDuration;
  if (deadline >= now) {
    return ProposalState.Active;
  } else if (!quorumReached(id) || !voteSucceeded(id)) {
    return ProposalState.Defeated;
  } else if (proposals[id].etaSeconds == 0) {
    return ProposalState.Succeeded;
  } else {
    return ProposalState.Queued;
  }
}
```

Recall that this is a widely-used *library contract*, not just some one-off project! As you can see, even just determining what state the proposal is in becomes exceedingly complex. Instead, it would be much simpler to describe a proposal starting from its states, then only include the fields relevant to each state. This is precisely what can be done with an ADT. As an additional benefit, the compiler rejects reading field values that are not relevant to the current state. For some additional clarity, here is what such an ADT might look like, in Rust syntax. It also demonstrates well that most fields are irrelevant for most states.

```
enum ProposalCore {
    Pending {
        voteStart: u64,
    },
    Active {
        voteStart: u64,
        voteDuration: u64,
    },
    Canceled,
    Defeated,
    Succeeded,
    Queued {
        etaSeconds: u64,
    },
    Expired,
    Executed,
}
```

Chapter 7

Conclusion

This page unintentionally left blank.

Bibliography

Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).